

# BT5110: Tutorial 2 — Entity - Relationship Model

Pratik Karmakar

School of Computing,  
National University of Singapore

AY26/27 S1



# The Problem

The Varsity International Network of Oenology wishes to computerise the management of the information about its members as well as to record the information they gather about various wines. Your company, Apasaja Private Limited, is commissioned by the Varsity International Network of Oenology to design and implement the relational schema of the database application. The organisation is big enough so that there could be several members with the same name. A card with a unique number is issued to identify each drinker. The contact address of each member is also recorded for the mailing of announcements and calls for meetings.

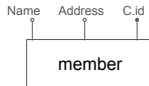
At most once a week, VINO organises a tasting session. At each session, the attending members taste several bottles. Each member records for each bottle his or her evaluation of the quality (very good, good, average, mediocre, bad, very bad) of each wine that she or he tastes. The evaluation may differ for the same wine from one drinker to another. Actual quality and therefore evaluation also varies from one to another bottle of a given wine. Every bottle that is opened during the tasting session is finished during that session.

Each wine is identified by its name ("Parade D'Amour"), appellation ("Bordeaux") and vintage (1990). Other information of interest about the wine is the degree of alcohol (11.5), where and by whom it has been bottled ("Mis en Bouteille par Amblard-Larolphe Negociant-Eleveur a Saint Andre de Cubzac (Gironde) - France"), the certification of its appellation if available ("Appellation Bordeaux Controlée"), and the country it comes from (produce of "France").

Generally, there are or have been several bottles of the same wine in the cellar. For each wine, the bottles in the wine cellar of VINO are numbered. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. For documentation purposes, VINO may also want to record wines for which it does not own bottles. The bottles are either available in the cellar or they have been tasted and emptied.

We first want to design an entity-relationship schema that most correctly and most completely captures the constraints expressed in the above description of the VINO application.

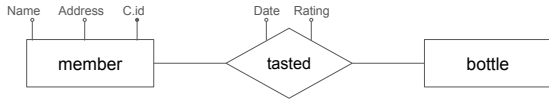
"The organisation is big enough so that there could be several **members** with the same **name**. A card with a **unique number** is issued to identify each **drinker**. The **contact address** of each member is also recorded for the mailing of announcements and calls for meetings."



**member** is an entity set with attributes: **Name**, **Address** and **Card Number (C.id)**, where C.id acts as the primary key.

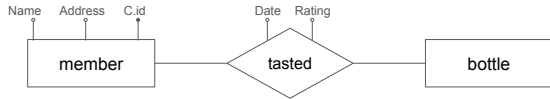
"At most once a week, VINO organises a tasting session. At each session, the **attending members** taste **several bottles**. Each member records for each bottle his or her **evaluation** of the quality (very good, good, average, mediocre, bad, very bad) of each wine that she or he tastes. The evaluation may differ for the same wine from one drinker to another. Actual quality and therefore evaluation also varies from one to another bottle of a given wine. Every bottle that is opened during the tasting session is finished during that session."

**bottle** is another entity set, and is related to **member** by a relationship called **tasted**: **Members taste bottles** in different sessions to give **ratings**.



"At most once a week, VINO organises a tasting session. At each session, the **attending members** taste **several bottles**. Each member records for each bottle his or her **evaluation** of the quality (very good, good, average, mediocre, bad, very bad) of each wine that she or he tastes. The evaluation may differ for the same wine from one drinker to another. Actual quality and therefore evaluation also varies from one to another bottle of a given wine. Every bottle that is opened during the tasting session is finished during that session."

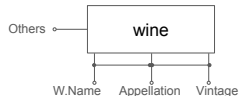
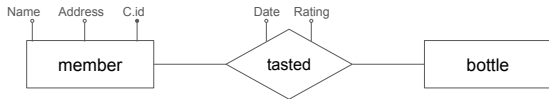
**bottle** is another entity set, and is related to **member** by a relationship called **tasted**: **Members taste bottles** in different sessions to give **ratings**.



We do not yet know the attributes of the **bottle** entity. But essentially necessary attributes from **member** and **bottle** will be borrowed by the relationship **tasted**.

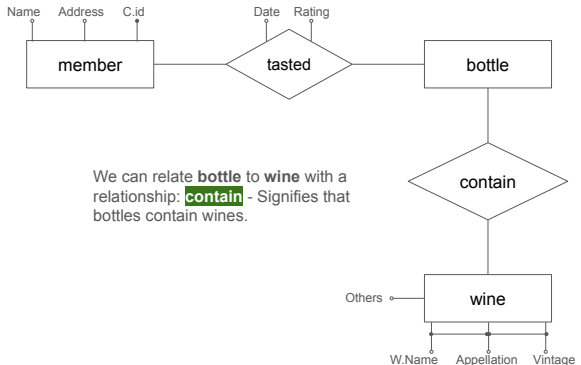
"Each **wine** is identified by its name ("Parade D'Amour"), **appellation** ("Bordeaux") and **vintage** (1990). Other information of interest about the wine is the **degree of alcohol** (11.5), where and **by whom it has been bottled** ("Mis en Bouteille par Amblard-Larolphe Negociant-Eleveur a Saint Andre de Cubzac (Gironde) - France"), the **certification of its appellation** if available ("Appellation Bordeaux Controlée"), and the **country it comes from** (produce of "France")."

**wine** is another entity set with a **composite primary key** which consists of **Name, Appellation & Vintage**. It has other attributes: Alcohol\_degree, Bottled\_by, Appellation\_cert and Country (denoted by **Others**).



"Each **wine** is identified by its name ("Parade D'Amour"), **appellation** ("Bordeaux") and **vintage** (1990). Other information of interest about the wine is the **degree of alcohol** (11.5), where and **by whom it has been bottled** ("Mis en Bouteille par Amblard-Larophie Negociant-Eleveur a Saint Andre de Cubzac (Gironde) - France"), the **certification of its appellation** if available ("Appellation Bordeaux Controlée"), and the **country it comes from** (produce of "France")."

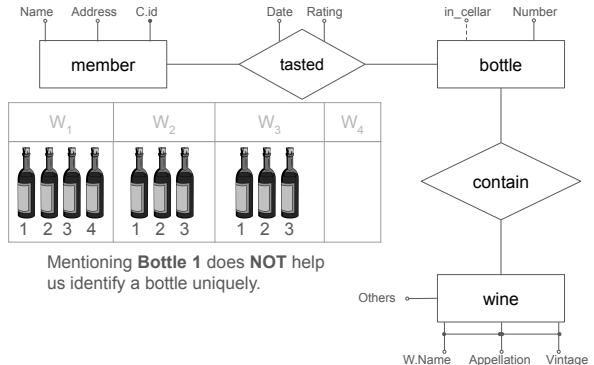
**wine** is another entity set with a **composite primary key** which consists of **Name, Appellation & Vintage**. It has other attributes: Alcohol\_degree, Bottled\_by, Appellation\_cert and Country (denoted by **Others**).



We can relate **bottle** to **wine** with a relationship: **contain** - Signifies that bottles contain wines.

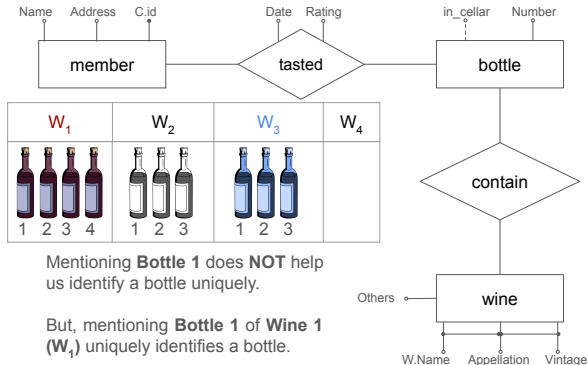
"Generally, there are or have been **several bottles of the same wine** in the cellar. For each wine, the bottles in the wine cellar of VINO are **numbered**. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. For documentation purposes, VINO may also want to record wines for which it does not own bottles. The bottles are either available in the cellar or they have been tasted and emptied."

**Bottles** have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.



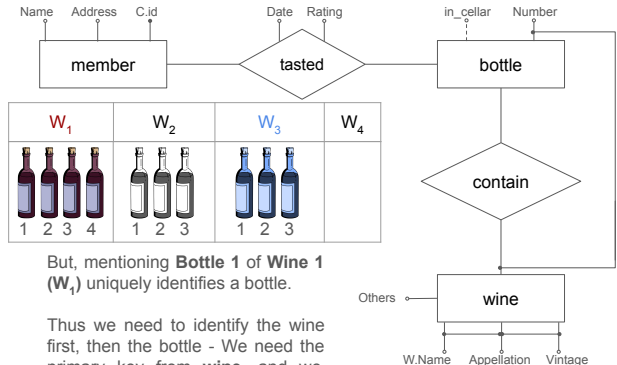
"Generally, there are or have been **several bottles of the same wine** in the cellar. For each wine, the bottles in the wine cellar of VINO are numbered. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. For documentation purposes, VINO may also want to record wines for which it does not own bottles. The bottles are either available in the cellar or they have been tasted and emptied."

**Bottles** have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.



"Generally, there are or have been **several bottles of the same wine** in the cellar. For each wine, the bottles in the wine cellar of VINO are **numbered**. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. For documentation purposes, VINO may also want to record wines for which it does not own bottles. The bottles are either available in the cellar or they have been tasted and emptied."

**Bottles** have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.

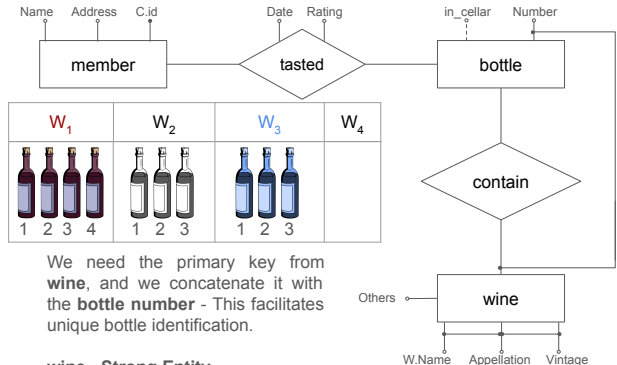


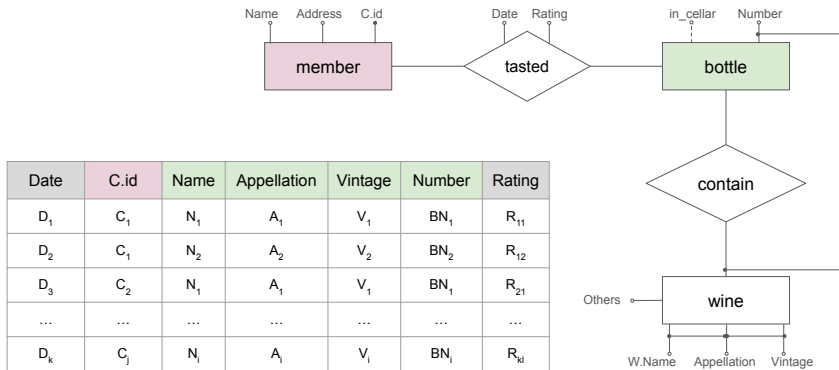
But, mentioning **Bottle 1 of Wine 1 ( $W_1$ )** uniquely identifies a bottle.

Thus we need to identify the wine first, then the bottle - We need the primary key from **wine**, and we concatenate it with the **bottle number** - This facilitates unique bottle identification.

"Generally, there are or have been **several bottles of the same wine** in the cellar. For each wine, the bottles in the wine cellar of VINO are **numbered**. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. For documentation purposes, VINO may also want to record wines for which it does not own bottles. The bottles are either available in the cellar or they have been tasted and emptied."

**Bottles** have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.

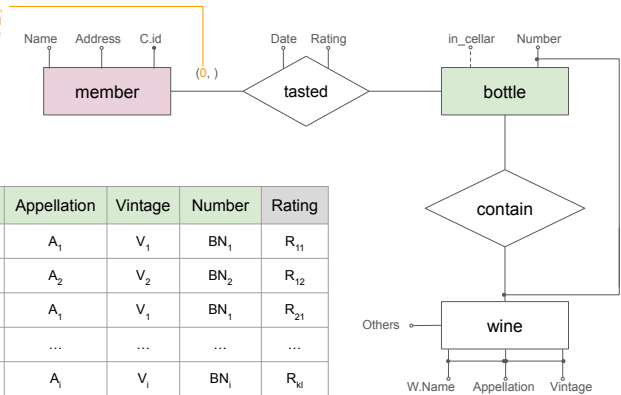




Example instantiation of the **tasted** relationship

There can be members who have **NOT** attended any session, thus have **NOT** tasted any bottle - Thus **DO NOT** appear in **Taste** relationship - Min constraint = 0

"At each session, **the attending members** taste several bottles". - Suggests that there can be non-attending members.

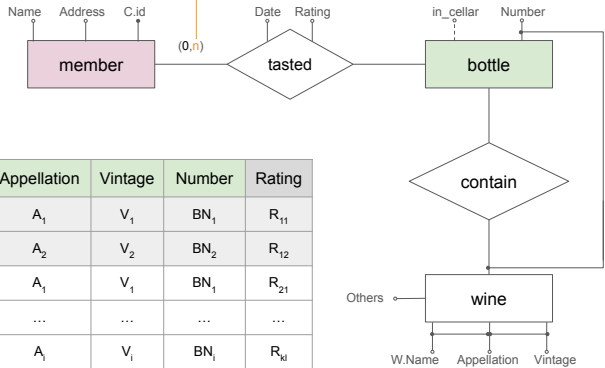


| Date           | C.id           | Name           | Appellation    | Vintage        | Number          | Rating          |
|----------------|----------------|----------------|----------------|----------------|-----------------|-----------------|
| D <sub>1</sub> | C <sub>1</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>11</sub> |
| D <sub>2</sub> | C <sub>1</sub> | N <sub>2</sub> | A <sub>2</sub> | V <sub>2</sub> | BN <sub>2</sub> | R <sub>12</sub> |
| D <sub>3</sub> | C <sub>2</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>21</sub> |
| ...            | ...            | ...            | ...            | ...            | ...             | ...             |
| D <sub>k</sub> | C <sub>j</sub> | N <sub>i</sub> | A <sub>i</sub> | V <sub>i</sub> | BN <sub>i</sub> | R <sub>ki</sub> |

Example instantiation of the **tasted** relationship

There can be members who have tasted more than one bottles - Thus can appear in Taste relationship **MORE THAN ONCE** - Max constraint = n (denotes **MANY**)

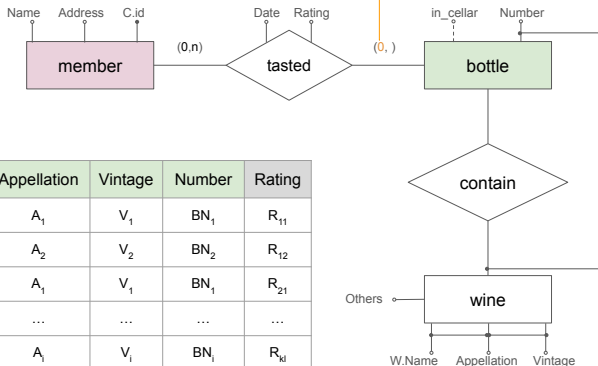
"At each session, the attending members **taste several bottles.**"  
- Suggests that there can be non-attending members.



| Date           | C.id           | Name           | Appellation    | Vintage        | Number          | Rating          |
|----------------|----------------|----------------|----------------|----------------|-----------------|-----------------|
| D <sub>1</sub> | C <sub>1</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>11</sub> |
| D <sub>2</sub> | C <sub>1</sub> | N <sub>2</sub> | A <sub>2</sub> | V <sub>2</sub> | BN <sub>2</sub> | R <sub>12</sub> |
| D <sub>3</sub> | C <sub>2</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>21</sub> |
| ...            | ...            | ...            | ...            | ...            | ...             | ...             |
| D <sub>k</sub> | C <sub>j</sub> | N <sub>i</sub> | A <sub>i</sub> | V <sub>i</sub> | BN <sub>i</sub> | R <sub>ki</sub> |

Example instantiation of the **tasted** relationship

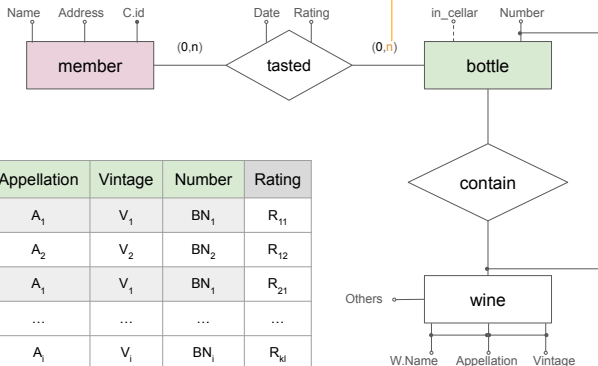
There can be bottles which **no member has tasted** - Thus they **DO NOT** appear in **Taste** relationship - Min constraint = 0



| Date           | C.id           | Name           | Appellation    | Vintage        | Number          | Rating          |
|----------------|----------------|----------------|----------------|----------------|-----------------|-----------------|
| D <sub>1</sub> | C <sub>1</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>11</sub> |
| D <sub>2</sub> | C <sub>1</sub> | N <sub>2</sub> | A <sub>2</sub> | V <sub>2</sub> | BN <sub>2</sub> | R <sub>12</sub> |
| D <sub>3</sub> | C <sub>2</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>21</sub> |
| ...            | ...            | ...            | ...            | ...            | ...             | ...             |
| D <sub>k</sub> | C <sub>j</sub> | N <sub>i</sub> | A <sub>i</sub> | V <sub>i</sub> | BN <sub>i</sub> | R <sub>ki</sub> |

Example instantiation of the **tasted** relationship

There can be bottles which **MANY** members have **tasted** - Thus they appear **MORE THAN ONCE** in Taste relationship - Max constraint = n (denotes **MANY**)

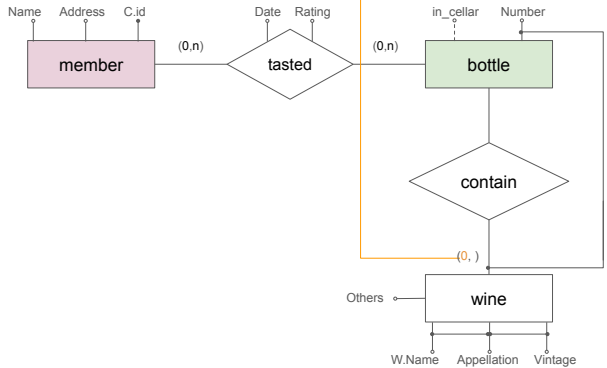


| Date           | C.id           | Name           | Appellation    | Vintage        | Number          | Rating          |
|----------------|----------------|----------------|----------------|----------------|-----------------|-----------------|
| D <sub>1</sub> | C <sub>1</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>11</sub> |
| D <sub>2</sub> | C <sub>1</sub> | N <sub>2</sub> | A <sub>2</sub> | V <sub>2</sub> | BN <sub>2</sub> | R <sub>12</sub> |
| D <sub>3</sub> | C <sub>2</sub> | N <sub>1</sub> | A <sub>1</sub> | V <sub>1</sub> | BN <sub>1</sub> | R <sub>21</sub> |
| ...            | ...            | ...            | ...            | ...            | ...             | ...             |
| D <sub>k</sub> | C <sub>j</sub> | N <sub>i</sub> | A <sub>i</sub> | V <sub>i</sub> | BN <sub>i</sub> | R <sub>ki</sub> |

Example instantiation of the **tasted** relationship

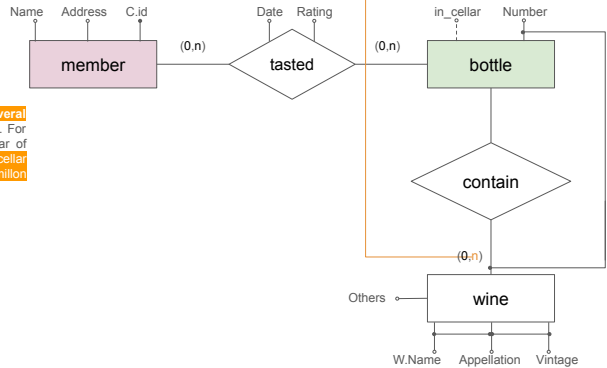
There can be wines, for which VINO has/had no bottles - Thus **DO NOT** appear in Contain relationship - Min constraint = 0

"For documentation purposes, VINO may also want to record wines for which it does not own bottles."

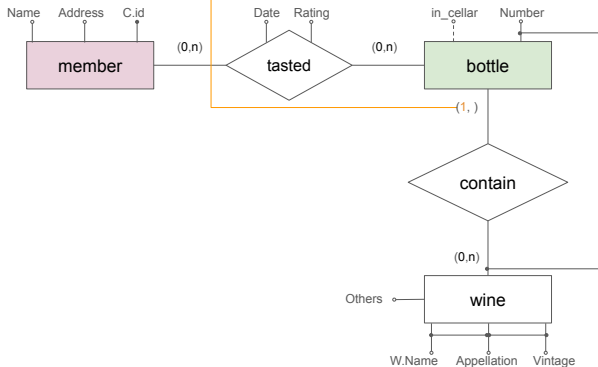


There can be **MANY** bottles of a wine - Thus can appear in Contain relationship **MORE THAN ONCE** - Max constraint = n(denotes **MANY**)

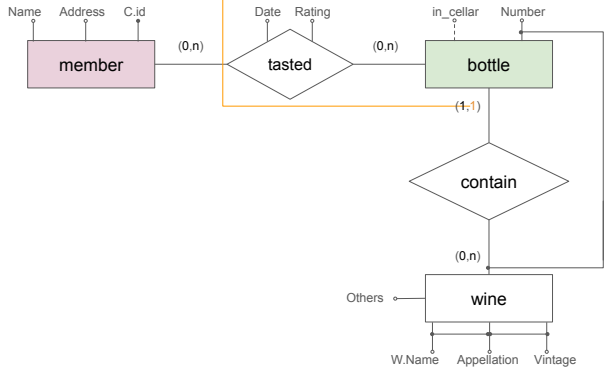
Generally, there are or have been several bottles of the same wine in the cellar. For each wine, the bottles in the wine cellar of VINO are numbered. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara.

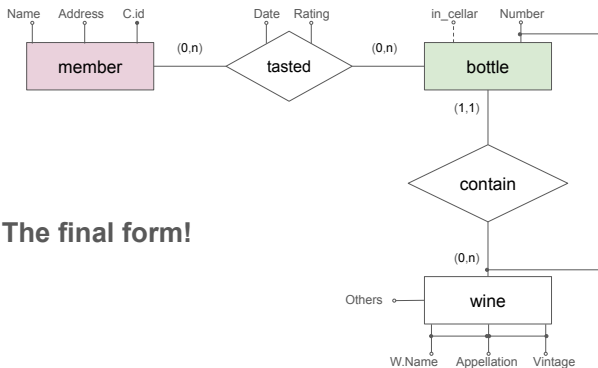


If there exists a bottle in the database, it has/had some wine - Thus it has to be in the Contain relationship - Min constraint = 1

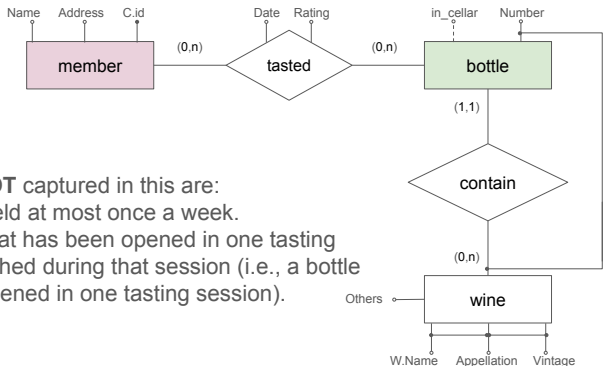


One bottle cannot contain more than one wines - Thus can appear in the Contain relationship only once - Max constraint = 1



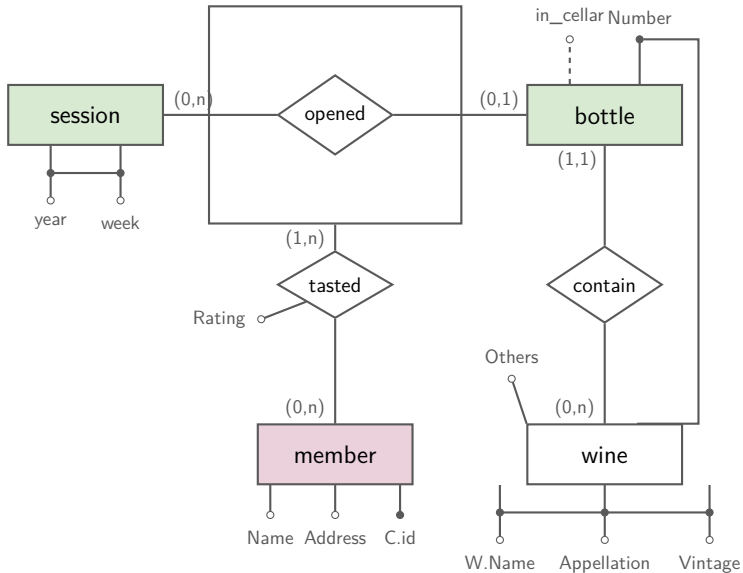


**The final form!**



The constraints **NOT** captured in this are:

- A session is held at most once a week.
- Every bottle that has been opened in one tasting session is finished during that session (i.e., a bottle can only be opened in one tasting session).



## Q2. Logical Design

Translate the final ER diagram (with session, open, and aggregation) into a relational schema.

- Strong entities member, wine, session: attributes carry over, entity key becomes the primary key.
- Weak entity bottle: primary key = its partial key (number) concatenated with wine's key.
- contain (bottle  $\rightarrow$  wine, (1,1)): FK on bottle, absorbed into its own row.
- open (session  $\leftrightarrow$  bottle, bottle side (0,1)): its own table, keyed by bottle.
- Aggregated taste (member  $\leftrightarrow$  open): FK references open, not bottle directly.

## Q2. Logical Design — member & wine

```
1 CREATE TABLE member (  
2     number CHAR(10) PRIMARY KEY,  
3     name    VARCHAR(32) NOT NULL,  
4     address VARCHAR(64) NOT NULL  
5 );  
6  
7 CREATE TABLE wine (  
8     name          VARCHAR(32),  
9     appellation   VARCHAR(32),  
10    vintage       INTEGER,  
11    alcohol_degree NUMERIC NOT NULL,  
12    bottled        VARCHAR(128) NOT NULL,  
13    certification  VARCHAR(64),  
14    country        VARCHAR(32) NOT NULL,  
15    PRIMARY KEY (name, appellation, vintage)  
16 );
```

## Q2. Logical Design — session & bottle

```
1 CREATE TABLE session (  
2     year INTEGER,  
3     week INTEGER,  
4     PRIMARY KEY (year, week)  
5 );  
6 CREATE TABLE bottle (  
7     wine_name  VARCHAR(32),  
8     appellation VARCHAR(32),  
9     vintage    INTEGER,  
10    number     INTEGER CHECK (number > 0),  
11    PRIMARY KEY (number, wine_name, appellation, vintage),  
12    FOREIGN KEY (wine_name, appellation, vintage)  
13        REFERENCES wine (name, appellation, vintage)  
14 );
```

**(1,1) on bottle's contain** → (wine\_name, appellation, vintage)  
is NOT NULL by construction of the PK.

## Q2. Logical Design — open

```
1 CREATE TABLE open (  
2     wine_name      VARCHAR(32),  
3     appellation    VARCHAR(32),  
4     vintage        INTEGER,  
5     bottle_number  INTEGER,  
6     session_year   INTEGER NOT NULL,  
7     session_week   INTEGER NOT NULL,  
8     PRIMARY KEY (bottle_number, wine_name, appellation, vintage),  
9     FOREIGN KEY (session_year, session_week)  
10        REFERENCES session (year, week),  
11     FOREIGN KEY (bottle_number, wine_name, appellation, vintage)  
12        REFERENCES bottle (number, wine_name, appellation, vintage)  
13 );
```

**Bottle (0,1) in open:** a row exists only for bottles that have been opened → open's PK = bottle's key (at most one session per bottle).

## Q2. Logical Design — taste (aggregate)

```
1 CREATE TABLE taste (  
2     wine_name      VARCHAR(32),  
3     appellation    VARCHAR(32),  
4     vintage        INTEGER,  
5     bottle_number  INTEGER,  
6     member         CHAR(10),  
7     rating         VARCHAR(32) NOT NULL,  
8     PRIMARY KEY (member, bottle_number, wine_name, appellation, vintage),  
9     FOREIGN KEY (member) REFERENCES member (number),  
10    FOREIGN KEY (bottle_number, wine_name, appellation, vintage)  
11        REFERENCES open (bottle_number, wine_name, appellation, vintage),  
12    CHECK (rating IN ('very good', 'good', 'average',  
13                  'mediocre', 'bad', 'very bad'))  
14 );
```

Referencing open (not bottle) is what the aggregation encodes: a rating is tied to a specific *opening event*, not just to a bottle.

Questions?

Drop a mail at: [pratik.karmakar@u.nus.edu](mailto:pratik.karmakar@u.nus.edu)